

Music Genre Classification with Deep Neural Networks

We follow <https://www.kdnuggets.com/2020/02/audio-data-analysis-deep-learning-python-part-1.html>.

See DFT, STFT, Spectrograms and MFCCs.ipynb and librosa Audio Feature Extraction.ipynb for review.

We use the *GTZAN Genre Collection* dataset, available at <http://marsyas.info/downloads/datasets.html>. Our task is to build a model that classifies a music excerpt into music genres.

In this notebook, we construct spectrograms from the audio (.wav) file and extract spectral features (see librosa Audio Feature Extraction.ipynb). The spectrograms are stored in PNG format and the spectral features are stored in CSV files. With the spectral features as inputs, we then construct an XGBoost model and a deep neural network model. In the next notebook, we will instead use the spectrogram image files as inputs, and construct a convolutional neural network for the task.

Workflow:

0. Global variables
1. Dataset: preliminary observations and pre-processing
2. Convert audio files into spectrograms, store in PNG format
3. Extract spectral features, store in CSV files
4. Train-validation split and pre-processing
5. XGBoost and a Deep neural network

Preamble

```
In [1]: import IPython.display as ipyd
        from tqdm import tqdm
        import matplotlib.pyplot as plt
        import numpy as np
        import seaborn as sns

        import pandas as pd
        # Make the output look better
        pd.set_option('display.max_rows', 500)
        pd.set_option('display.max_columns', 500)
        # pd.set_option('display.width', 1000)
        pd.options.mode.chained_assignment = None # default='warn' # ignores warning about dropping columns inplace

        from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import MinMaxScaler
        from sklearn.preprocessing import OneHotEncoder

        from xgboost import XGBClassifier

        from scipy import signal
        from scipy.io import wavfile # Load wave files
        import librosa # package for music and audio analysis
        import librosa.display

        from PIL import Image # Load image

        import os
        os.chdir(r'C:\Users\Cedric Yu\Desktop\Data Science\Audio Data Analysis\1_GTZAN_genre_classification')
```

Global variables

```
In [19]: num_class = 10

        # raw audio file and STFT parameters
        mono = True # mono channel
        duration = 30 # 30 seconds
        sr = 22050 # sample rate = 22050 Hz
```

```

n_fft = 551      # n_fft = 0.025 ms * 22050 Hz
hop_length = 220  # hop_length = 0.01 ms * 22050 Hz

# parameters for the spectral features
roll_percent = 0.85 # roll-off percentage in spectral rolloff
n_mfcc = 20 # number of MFCCs to return

# training set size
train_size = 0.8

```

1. Dataset: preliminary observations and pre-processing

Metadata

Our dataset, *GTZAN Genre Collection*, is downloaded from <http://marsyas.info/downloads/datasets.html>. The dataset is 1.23GB after extraction. It consists of 1000 audio tracks, each of 30 seconds long. It contains 10 genres, in each of which there are 100 tracks. The tracks are all 22050Hz Mono 16-bit audio files in .wav format.

The 10 genres are:

1. Blues
2. Classical
3. Country
4. Disco
5. Hiphop
6. Jazz
7. Metal
8. Pop
9. Reggae
10. Rock

Each genre has its own directory storing the audio tracks to which they belong:



Get genre and file names

```

In [16]: # List of genres
genres_list = os.listdir('genres/')
genres_list.sort()

```

```

In [17]: genres_encode = dict(zip(genres_list, range(len(genres_list))))
genres_decode = dict(zip(range(len(genres_list)), genres_list))

```

```

In [18]: genres_list

```

```

Out[18]: ['blues',
'classical',
'country',
'disco',
'hiphop',
'jazz',
'metal',
'pop',
'reggae',
'rock']

```

```

In [86]: genres_encode

```

```

Out[86]: {'blues': 0,
'classical': 1,
'country': 2,
'disco': 3,

```

```
'hiphop': 4,  
'jazz': 5,  
'metal': 6,  
'pop': 7,  
'reggae': 8,  
'rock': 9}
```

```
In [88]: genres_decode
```

```
Out[88]: {0: 'blues',  
1: 'classical',  
2: 'country',  
3: 'disco',  
4: 'hiphop',  
5: 'jazz',  
6: 'metal',  
7: 'pop',  
8: 'reggae',  
9: 'rock'}
```

```
In [4]: # wav_filenames is a dict that stores the list of file names in each genre folder  
wav_filenames = {}  
for genre in genres_list:  
    wav_filenames[genre] = os.listdir('genres/'+genre+'/')  
    wav_filenames[genre].sort()
```

Sample audio file

Load audio file

```
In [6]: filename_sample = 'genres/' + 'classical' + '/' + wav_filenames['classical'][10]
```

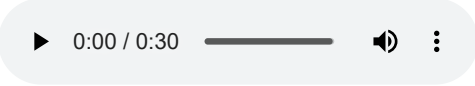
```
In [7]: filename_sample
```

```
Out[7]: 'genres/classical/classical.00010.wav'
```

```
In [8]: wav_filenames['classical'][10][:4]
```

```
Out[8]: 'classical.00010'
```

```
In [9]: # use librosa.load() to load audio file  
# set duration to exactly 30 seconds  
x_sample, sr_sample = librosa.load(filename_sample, sr=None, mono=True, duration=30)  
# x, sr = librosa.load(filename, sr=None, mono=True, offset=0.1)  
  
# play audio on IPython  
ipyd.Audio(rate=sr_sample, data=x_sample)
```

```
Out[9]: 
```

```
In [10]: print(x_sample)  
print(sr_sample)
```

```
[0.01327515 0.02868652 0.03347778 ... 0.03329468 0.03674316 0.03027344]  
22050
```

```
In [11]: x_sample.shape
```

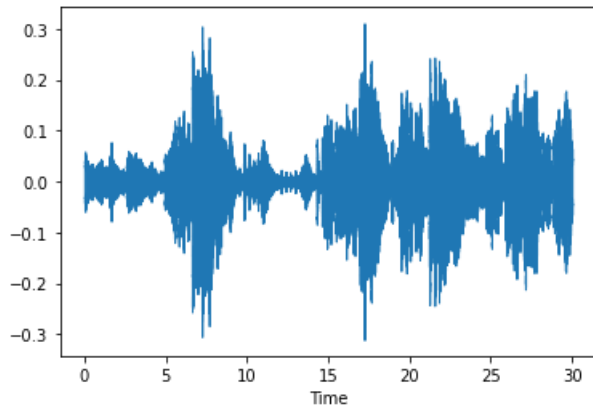
```
Out[11]: (661500,)
```

```
In [12]: x_sample.shape[0]/sr_sample
```

Out[12]: 30.0

Waveform plot

```
In [13]: librosa.display.waveshow(x_sample, sr=sr_sample);
```



Spectrogram

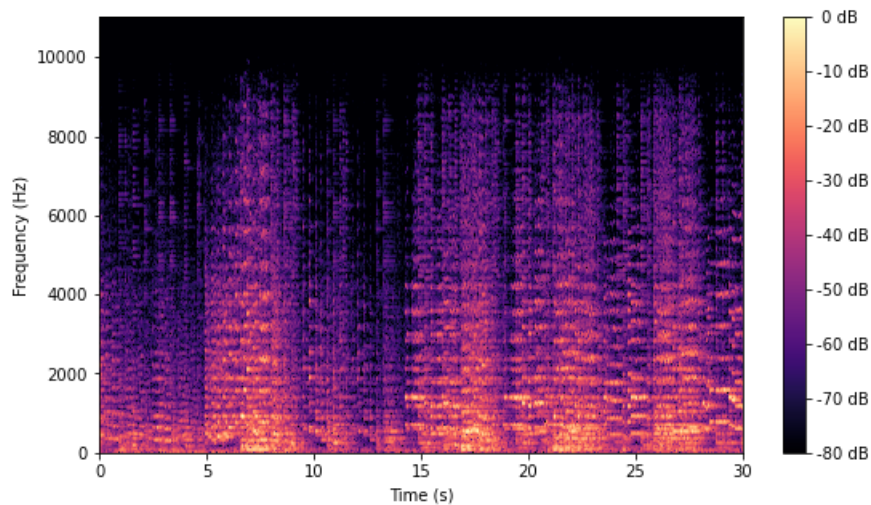
```
In [14]: # Short-time fast Fourier transform (STFFT)
n_fft_sample = int(0.025*sr_sample) # 25ms
hop_length_sample = int(0.01*sr_sample) # 10ms
stfft_sample = librosa.stft(x_sample, n_fft=n_fft_sample, hop_length=hop_length_sample)
abs_stfft_sample = np.abs(stfft_sample)

stftdB_sample = librosa.amplitude_to_db(abs_stfft_sample, ref=np.max)
```

```
In [15]: stftdB_sample.shape
```

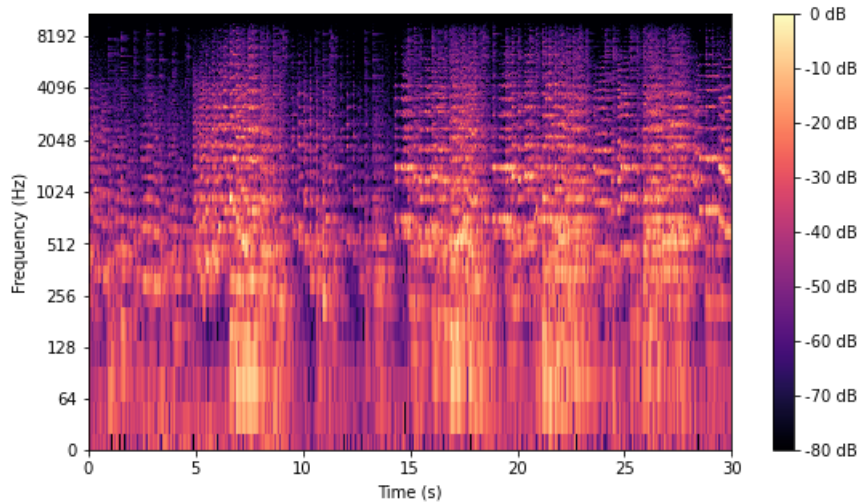
Out[15]: (276, 3007)

```
In [16]: # Plot spectrogram
fig, ax = plt.subplots(figsize=(9,5))
img_sample = librosa.display.specshow(stftdB_sample, sr=sr_sample, hop_length=hop_length_sample,
                                     y_axis='hz', x_axis='time', ax=ax)
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
# plt.ylim([0, 1800])
fig.colorbar(img_sample, ax=ax, format="%2.0f dB");
```

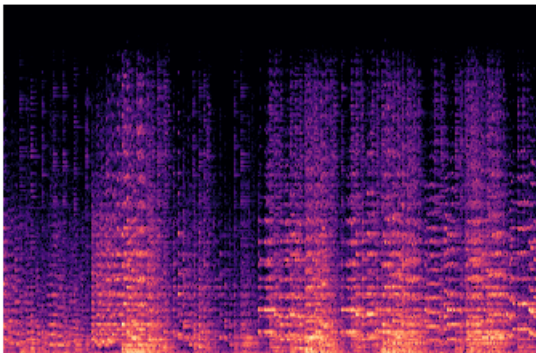


```
In [17]: # Plot spectrogram, frequency in Log scale
fig, ax = plt.subplots(figsize=(9,5))
```

```
img_sample = librosa.display.specshow(stfttdB_sample, sr=sr_sample, hop_length=hop_length_sample,
                                     y_axis='log', x_axis='time', ax=ax)
plt.xlabel('Time (s)')
plt.ylabel("Frequency (Hz)")
# plt.ylim([0, 1800])
fig.colorbar(img_sample, ax=ax, format="%2.0f dB");
```



```
In [18]: # Plot spectrogram, without x- and y-axes and without white margins
# Plots like this will be stored in PNG format for future use
img_sample = librosa.display.specshow(stfttdB_sample, sr=sr_sample, hop_length=hop_length_sample,
                                     y_axis='hz', x_axis='time')
plt.axis('off')
plt.savefig('spectrogram_sample.png', bbox_inches='tight', pad_inches=0)
# plt.close()
```



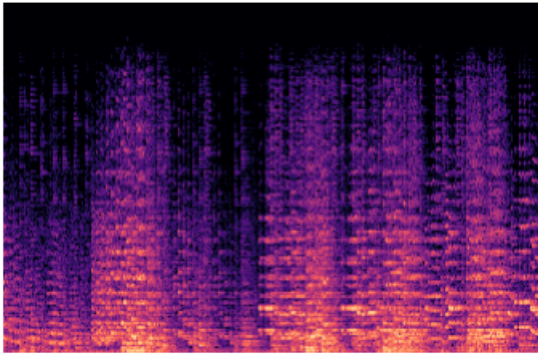
```
In [19]: os.path.exists('spectrograms/' + 'rock')
```

Out[19]: True

```
In [134... os.makedirs('spectrograms/' + 'rock')
```

```
In [21]: im_sample = Image.open('spectrogram_sample.png')
```

```
In [22]: plt.imshow(im_sample);
plt.axis('off');
```



2. Convert audio files into spectrograms, store in PNG format

```
In [63]: # construct spectrograms and save to PNG files

for genre in tqdm(genres_list):

    # create destination directories for output PNG files if not exist
    if not os.path.exists('spectrograms/' + genre):
        os.makedirs('spectrograms/' + genre)

    filenames = wav_filenames[genre]

    for wav_filename in filenames:

        # use librosa.load() to load audio file
        filename = 'genres/' + genre + '/' + wav_filename
        x, _ = librosa.load(filename, sr=sr, mono=mono, duration=duration)

        # short-time fast Fourier transform (STFFT)
        stfft = librosa.stft(x, n_fft=n_fft, hop_length=hop_length)
        abs_stfft = np.abs(stfft)
        # convert amplitude to dB scale
        stftdB = librosa.amplitude_to_db(abs_stfft, ref=np.max)

        # Plot spectrogram, without x- and y-axes
        # Plots like this will be stored in PNG format for future use
        img = librosa.display.specshow(stftdB, sr=sr, hop_length=hop_length,
                                         y_axis='hz', x_axis='time')

        plt.axis('off');
        # save to PNG file
        plt.savefig('spectrograms/' + genre + '/' + wav_filename[:-4] + '.png',
                    bbox_inches='tight', pad_inches=0) # no white margins
        plt.close()
```

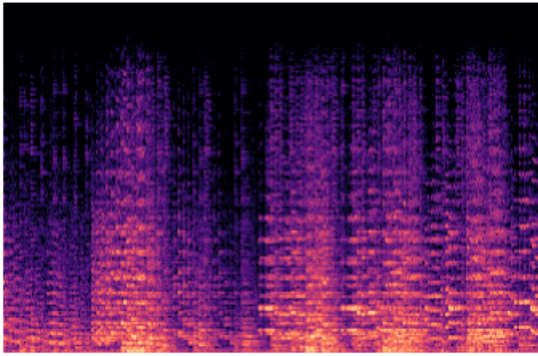
100%|██████████| 10/10 [08:21<00:00, 50.14s/it]

```
In [25]: # create dictionary for spectrogram_filenames that stores the list of file names in each genre folder
spectrogram_filenames = {}
for genre in genres_list:
    spectrogram_filenames[genre] = os.listdir('spectrograms/'+genre+'/')
    spectrogram_filenames[genre].sort()
```

```
In [26]: im_sample = Image.open('spectrograms/' + 'blues' + '/' + spectrogram_filenames['blues'][3])
```

```
In [27]: print('spectrograms/' + 'blues' + '/' + spectrogram_filenames['blues'][3])
plt.imshow(im_sample);
plt.axis('off');
```

spectrograms/blues/blues.00003.png



In []:

In []:

3. Extract spectral features, store in CSV files

See librosa Audio Feature Extraction.ipynb.

We will extract the following spectral features (can consider more if you want):

1. Spectral Centroid
2. Spectral Rolloff
3. Spectral Bandwidth
4. Zero-Crossing Rate
5. RMS energy (RMSE)
6. Mel-Frequency Cepstral Coefficients(MFCCs)
7. Chroma feature

More precisely, for each audio file, we will only use the mean value of each of these features. For MFCC and chroma feature, we will take the mean of each of the 20 MFCCs and the 12 tones.

Sample audio file

```
In [29]: filename_sample = 'genres/' + 'classical' + '/' + wav_filenames['classical'][10]
```

```
In [30]: # use librosa.load() to load audio file
# set duration to exactly 30 seconds
x_sample, _ = librosa.load(filename_sample, sr=sr, mono=mono, duration=duration)
# x, sr = librosa.load(filename, sr=None, mono=True, offset=0.1)

# play audio on IPython
ipyd.Audio(rate=sr_sample, data=x_sample)
```

Out[30]:

```
In [31]: # Short-time fast Fourier transform (STFFT)
stfft_sample = librosa.stft(x_sample, n_fft=n_fft, hop_length=hop_length)
abs_stfft_sample = np.abs(stfft_sample)

stfttdB_sample = librosa.amplitude_to_db(abs_stfft_sample, ref=np.max)
```

```
In [32]: stfttdB_sample.shape
```

```
Out[32]: (276, 3007)
```

```
In [33]: t_sample = librosa.frames_to_time(frames=np.arange(stft_sample.shape[1]), sr=sr,
          hop_length=hop_length, n_fft=n_fft)
```

1. Spherical centroids

```
In [34]: spectral_centroids_sample = librosa.feature.spectral_centroid(x_sample, sr=sr, n_fft=n_fft, hop_length=hop_length)
```

```
In [35]: spectral_centroids_sample.shape
```

```
Out[35]: (1, 3007)
```

```
In [57]: spectral_centroids_sample
```

```
Out[57]: array([[2002.25950539, 1554.30489019, 1603.11560928, ..., 1599.31353774,
                1555.59989945, 1629.9180745 ]])
```

2. Spectral rolloff

```
In [36]: spectral_rolloff_sample = librosa.feature.spectral_rolloff(x_sample, sr=sr, n_fft=n_fft, hop_length=hop_length, r
```

```
In [38]: spectral_rolloff_sample.shape
```

```
Out[38]: (1, 3007)
```

```
In [56]: spectral_rolloff_sample
```

```
Out[56]: array([[3327.54545455, 2726.18181818, 3046.90909091, ..., 3006.81818182,
                3006.81818182, 3407.72727273]])
```

3. Order-2 spectral bandwidth

```
In [39]: spectral_bandwidth_2_sample = librosa.feature.spectral_bandwidth(x_sample, sr=sr,
          n_fft=n_fft, hop_length=hop_length,
          p=2)
```

```
In [40]: spectral_bandwidth_2_sample.shape
```

```
Out[40]: (1, 3007)
```

```
In [55]: spectral_bandwidth_2_sample
```

```
Out[55]: array([[2046.94160518, 1478.35741311, 1429.00870858, ..., 1593.36052155,
                1660.4330939 , 1762.60892256]])
```

4. Zero-Crossing Rate (ZCR)

```
In [41]: zcr_sample = librosa.feature.zero_crossing_rate(x_sample, frame_length=n_fft, hop_length=hop_length)
```

```
In [42]: zcr_sample.shape
```

```
Out[42]: (1, 3007)
```

```
In [54]: zcr_sample
```

```
Out[54]: array([[0.04900181, 0.09618875, 0.11978221, ..., 0.10344828, 0.09437387,
                0.07803993]])
```

5. RMS Energy (RMSE)

```
In [48]: rmse_sample = librosa.feature.rms(x_sample, frame_length=n_fft, hop_length=hop_length)
```

```
In [49]: rmse_sample.shape
```

```
Out[49]: (1, 3007)
```

```
In [52]: rmse_sample
```

```
Out[52]: array([[0.01888934, 0.0193143 , 0.02063283, ..., 0.03095325, 0.02892459,
                  0.02414735]], dtype=float32)
```

```
In [62]: np.mean(rmse_sample)
```

```
Out[62]: 0.029104656
```

6. Mel-Frequency Cepstral Coefficients(MFCCs)

```
In [43]: mfcc_sample = librosa.feature.mfcc(x_sample, sr=sr, n_mfcc=20)
```

```
In [44]: mfcc_sample.shape
```

```
Out[44]: (20, 1292)
```

```
In [53]: mfcc_sample
```

```
Out[53]: array([[ -2.88814270e+02, -3.03552734e+02, -3.08814911e+02, ...,
                -2.90010529e+02, -2.87709595e+02, -2.86810181e+02],
               [ 1.20404900e+02, 1.30336243e+02, 1.31296692e+02, ...,
                1.03445786e+02, 1.07325943e+02, 1.17916489e+02],
               [-3.25257263e+01, -3.57221756e+01, -4.95719986e+01, ...,
                -3.97588120e+01, -3.74285927e+01, -3.13870716e+01],
               ...,
               [-9.52418041e+00, -1.28159914e+01, -7.07966137e+00, ...,
                -3.00743580e-01, -8.12984109e-01, 1.09867430e+00],
               [-2.52315216e+01, -2.32383251e+01, -1.48475132e+01, ...,
                1.24831653e+00, 3.84304476e+00, 7.88027573e+00],
               [-1.62425919e+01, -1.46614170e+01, -1.42571316e+01, ...,
                -9.76792145e+00, -5.51157951e+00, -1.23785651e+00]], dtype=float32)
```

```
In [60]: np.mean(mfcc_sample, axis=-1).shape
```

```
Out[60]: (20,)
```

7. Chroma feature

```
In [45]: chromagram_sample = librosa.feature.chroma_stft(x_sample, sr=sr, n_fft=n_fft, hop_length=hop_length)
```

```
In [46]: chromagram_sample.shape
```

```
Out[46]: (12, 3007)
```

```
In [51]: chromagram_sample
```

```
Out[51]: array([[0.7545146 , 0.3632676 , 0.55950844, ..., 0.05780026, 0.08090748,
                  0.12015166],
                [0.5536378 , 0.45821992, 0.71504474, ..., 0.4573516 , 0.39038655,
                  0.4222245 ],
                [0.12840906, 0.05570234, 0.08575641, ..., 1.          , 1.          ,
                  1.          ],
                ...])
```

```
[0.8112468 , 1.          , 1.          , ..., 0.0535132 , 0.04228062,
 0.0702644 ],
[0.4951272 , 0.46663687, 0.55220133, ..., 0.02690415, 0.03960161,
 0.05604427],
[0.75345343, 0.24317946, 0.32645005, ..., 0.02440774, 0.03550843,
 0.04975867]], dtype=float32)
```

```
In [61]: np.mean(chromagram_sample, axis=-1).shape
```

```
Out[61]: (12,)
```

Extract features and assign labels, store in pandas DataFrame, export to CSV

```
In [79]: # extract spectral features and store in pandas DataFrame

# filename will be used as index of the DataFrame
filenames_list = []
# column names
spectral_feature_names = ['spectral_centroids', 'spectral_rolloff', 'spectral_bandwidth',
                          'zcr', 'rmse']

for i in range(n_mfcc):
    spectral_feature_names.append('mfcc_'+str(i))
for j in range(12):
    spectral_feature_names.append('chroma_'+str(j))

spectral_features_list = []
# for feature in spectral_feature_names:
#     spectral_features_dict[feature] = []

target_list = []

for genre in tqdm(genres_list):

    filenames = wav_filenames[genre]

    for wav_filename in filenames:

        # use the filename as index, without the '.wav'
        filenames_list.append(wav_filename[:-4])

        # use librosa.load() to load audio file
        filename = 'genres/' + genre + '/' + wav_filename
        x, _ = librosa.load(filename, sr=sr, mono=mono, duration=duration)

        # spectral features
        current_feature_list = [] # stores feature values of current file
        spectral_centroids = librosa.feature.spectral_centroid(x, sr=sr,
                                                              n_fft=n_fft,
                                                              hop_length=hop_length)
        current_feature_list.append(np.mean(spectral_centroids))

        spectral_rolloff = librosa.feature.spectral_rolloff(x, sr=sr,
                                                           n_fft=n_fft,
                                                           hop_length=hop_length,
                                                           roll_percent=roll_percent)
        current_feature_list.append(np.mean(spectral_rolloff))

        spectral_bandwidth = librosa.feature.spectral_bandwidth(x, sr=sr,
                                                                n_fft=n_fft,
                                                                hop_length=hop_length,
                                                                p=2)
        current_feature_list.append(np.mean(spectral_bandwidth))

        zcr = librosa.feature.zero_crossing_rate(x, frame_length=n_fft, hop_length=hop_length)
        current_feature_list.append(np.mean(zcr))

        rmse = librosa.feature.rms(x, frame_length=n_fft, hop_length=hop_length)
        current_feature_list.append(np.mean(rmse))

        mfcc = librosa.feature.mfcc(x, sr=sr, n_mfcc=n_mfcc)
        mfcc_mean = np.mean(mfcc, axis=-1).squeeze()
        for i in range(n_mfcc):
```

```

        current_feature_list.append(mfcc_mean[i])

    chromagram = librosa.feature.chroma_stft(x, sr=sr, n_fft=n_fft, hop_length=hop_length)
    chromagram_mean = np.mean(chromagram, axis=-1).squeeze()
    for j in range(12):
        current_feature_list.append(chromagram_mean[j])

    spectral_features_list.append(current_feature_list)

    target_list.append(genres_encode[genre])

spectral_features_list = np.array(spectral_features_list)

```

100%|██████████| 10/10 [04:35<00:00, 27.55s/it]

```

In [91]: # store in dataframes
df_spectral_features = pd.DataFrame(spectral_features_list, columns = spectral_feature_names, index = filenames_list)
df_target = pd.DataFrame(target_list, columns = ['target'], index = filenames_list)

```

```

In [92]: df_spectral_features_target = pd.concat([df_spectral_features, df_target], axis = 1)

```

```

In [97]: df_spectral_features_target.iloc[:1000:50]

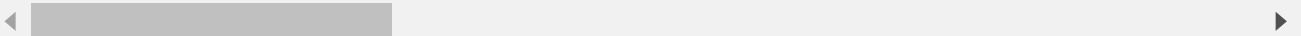
```

```

Out[97]:

```

	spectral_centroids	spectral_rolloff	spectral_bandwidth	zcr	rmse	mfcc_0	mfcc_1	mfcc_2	mfcc
blues.00000	1675.252460	3531.999758	1925.906422	0.083004	0.126734	-113.596748	121.557297	-19.158825	42.3510
blues.00050	1825.629671	3932.415546	2030.930730	0.085554	0.177232	-82.979019	107.052124	-25.320452	57.1249
classical.00000	1451.772666	2559.471899	1502.522797	0.098234	0.035813	-268.155945	135.689575	-34.070629	25.6308
classical.00050	1074.523477	2099.673036	1486.175141	0.049445	0.007923	-449.595581	151.507782	-12.747926	44.2706
country.00000	2478.190928	5600.887989	2737.508432	0.106933	0.247840	-23.153112	70.263184	3.823980	28.3301
country.00050	1621.016973	3460.524080	1921.704686	0.078904	0.108166	-143.093948	121.893066	-13.339254	47.7677
disco.00000	2080.481095	4271.488376	2392.117284	0.096389	0.136521	-111.191055	94.034180	12.493180	16.7507
disco.00050	2236.453183	4702.742298	2178.707161	0.128720	0.155485	-45.783978	101.753555	-32.084698	34.2762
hiphop.00000	2496.304621	5135.542794	2277.568647	0.147123	0.117197	-80.906776	84.792664	-31.550186	33.9019
hiphop.00050	1426.049832	2955.061311	2225.624847	0.042485	0.265753	-114.022079	110.130524	17.934523	37.9334
jazz.00000	1430.507076	2635.147323	1672.682284	0.078423	0.045760	-251.967300	137.889999	-11.156485	8.3573
jazz.00050	2284.716744	5407.806331	2685.651096	0.080768	0.189033	-55.165470	77.474060	8.271832	31.5641
metal.00000	3123.861426	6170.960184	2686.496517	0.182891	0.115272	-52.901485	50.085773	-11.487131	46.3499
metal.00050	2850.593026	5474.908940	2383.679872	0.177240	0.245179	16.518791	71.174026	-22.344831	60.2830
pop.00000	3070.522754	7366.794540	3297.534172	0.126879	0.180697	-94.529480	55.650463	29.270590	27.0637
pop.00050	2699.863243	5882.950933	2954.247120	0.118316	0.202376	-78.168694	65.982140	24.572762	25.6505
reggae.00000	1930.328540	4241.610182	2155.442081	0.097792	0.091142	-159.453445	104.844696	-10.300993	34.6618
reggae.00050	2509.478561	5059.774042	2490.255814	0.126931	0.125669	-104.857368	66.384926	-2.305362	24.6086
rock.00000	1851.466076	3943.361550	2029.242307	0.090265	0.127316	-116.608017	109.269157	-23.147902	40.6912
rock.00050	1491.342112	3180.021042	2118.870998	0.053392	0.135721	-144.985992	118.478111	20.531466	28.5704



```

In [98]: # save to CSV files
df_spectral_features_target.to_csv('df_spectral_features_target.csv')
df_spectral_features.to_csv('df_spectral_features.csv')
df_target.to_csv('df_target.csv')

```

In []:

In []:

4. Train-validation split and pre-processing

Train-validation split

In [4]:

```
df_spectral_features_target = pd.read_csv('df_spectral_features_target.csv', index_col = [0])
```

In [5]:

```
df_spectral_features_target = df_spectral_features_target.reset_index().drop(['index'], axis=1)
```

In [6]:

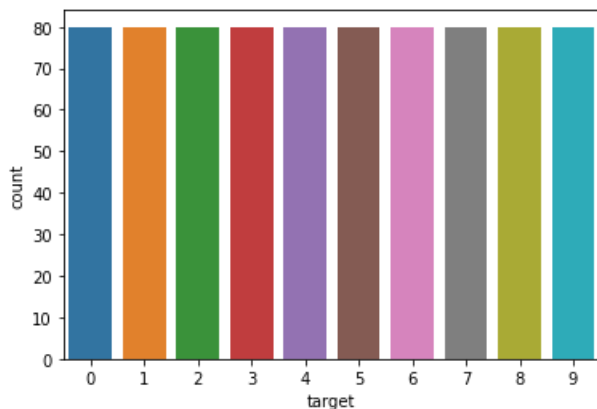
```
df_spectral_features = df_spectral_features_target.drop('target', axis=1)
df_target = df_spectral_features_target[['target']]
```

In [7]:

```
# train-validation split stratified by target label
X_train, X_valid, y_train, y_valid = train_test_split(df_spectral_features,
                                                    df_target,
                                                    train_size = train_size,
                                                    stratify=df_target,
                                                    random_state=42)
```

In [8]:

```
sns.countplot(x='target', data=y_train);
```



In [9]:

```
print(X_train.index[:10])
print(y_train.index[:10])
```

```
Int64Index([575, 204, 964, 503, 502, 498, 210, 778, 808, 906], dtype='int64')
Int64Index([575, 204, 964, 503, 502, 498, 210, 778, 808, 906], dtype='int64')
```

Pre-processing: min-max scaling

In [10]:

```
# min-max scaler
scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_valid_scaled = scaler.transform(X_valid)
```

5. XGBoost and Deep neural network

In [3]:

```
from sklearn.metrics import accuracy_score
```

XGBoost

```
In [174... XGBC_model = XGBClassifier(eval_metric='merror', objective='multi:softmax', n_jobs = 6,
                             max_depth=4, max_bin=128, reg_lambda=0.1
                             )
# 'merror' = multiclass classification error rate. It is calculated as #(wrong cases)/#(all cases)
```

```
In [175... XGBC_model.fit(X_train_scaled, y_train,
                  eval_set = [(X_train_scaled, y_train),
                              (X_valid_scaled, y_valid)])
```

[0]	validation_0-merror:0.30125	validation_1-merror:0.565
[1]	validation_0-merror:0.26	validation_1-merror:0.52
[2]	validation_0-merror:0.21625	validation_1-merror:0.515
[3]	validation_0-merror:0.19625	validation_1-merror:0.49
[4]	validation_0-merror:0.17375	validation_1-merror:0.485
[5]	validation_0-merror:0.16625	validation_1-merror:0.49
[6]	validation_0-merror:0.14625	validation_1-merror:0.505
[7]	validation_0-merror:0.13625	validation_1-merror:0.485
[8]	validation_0-merror:0.1275	validation_1-merror:0.485
[9]	validation_0-merror:0.12875	validation_1-merror:0.46
[10]	validation_0-merror:0.12125	validation_1-merror:0.46
[11]	validation_0-merror:0.115	validation_1-merror:0.465
[12]	validation_0-merror:0.10625	validation_1-merror:0.47

C:\ProgramData\Anaconda3\lib\site-packages\sklearn\utils\validation.py:63: DataConversionWarning: A column-vector y was passed when a 1d array was expected. Please change the shape of y to (n_samples,), for example using ravel().

```
return f(*args, **kwargs)
```

[13]	validation_0-merror:0.0975	validation_1-merror:0.475
[14]	validation_0-merror:0.0875	validation_1-merror:0.47
[15]	validation_0-merror:0.0825	validation_1-merror:0.465
[16]	validation_0-merror:0.07875	validation_1-merror:0.455
[17]	validation_0-merror:0.07375	validation_1-merror:0.45
[18]	validation_0-merror:0.07	validation_1-merror:0.45
[19]	validation_0-merror:0.06375	validation_1-merror:0.465
[20]	validation_0-merror:0.0625	validation_1-merror:0.445
[21]	validation_0-merror:0.055	validation_1-merror:0.435
[22]	validation_0-merror:0.0525	validation_1-merror:0.43
[23]	validation_0-merror:0.04875	validation_1-merror:0.42
[24]	validation_0-merror:0.04375	validation_1-merror:0.43
[25]	validation_0-merror:0.04375	validation_1-merror:0.43
[26]	validation_0-merror:0.03875	validation_1-merror:0.42
[27]	validation_0-merror:0.035	validation_1-merror:0.41
[28]	validation_0-merror:0.0325	validation_1-merror:0.405
[29]	validation_0-merror:0.03	validation_1-merror:0.405
[30]	validation_0-merror:0.025	validation_1-merror:0.405
[31]	validation_0-merror:0.02375	validation_1-merror:0.405
[32]	validation_0-merror:0.02125	validation_1-merror:0.4
[33]	validation_0-merror:0.01875	validation_1-merror:0.4
[34]	validation_0-merror:0.02	validation_1-merror:0.395
[35]	validation_0-merror:0.01875	validation_1-merror:0.395
[36]	validation_0-merror:0.01875	validation_1-merror:0.39
[37]	validation_0-merror:0.01625	validation_1-merror:0.39
[38]	validation_0-merror:0.01625	validation_1-merror:0.39
[39]	validation_0-merror:0.015	validation_1-merror:0.4
[40]	validation_0-merror:0.01375	validation_1-merror:0.395
[41]	validation_0-merror:0.01125	validation_1-merror:0.385
[42]	validation_0-merror:0.01125	validation_1-merror:0.39
[43]	validation_0-merror:0.01125	validation_1-merror:0.395
[44]	validation_0-merror:0.00875	validation_1-merror:0.385
[45]	validation_0-merror:0.00875	validation_1-merror:0.39
[46]	validation_0-merror:0.00875	validation_1-merror:0.385
[47]	validation_0-merror:0.00875	validation_1-merror:0.385
[48]	validation_0-merror:0.00875	validation_1-merror:0.39
[49]	validation_0-merror:0.00875	validation_1-merror:0.39
[50]	validation_0-merror:0.005	validation_1-merror:0.395
[51]	validation_0-merror:0.005	validation_1-merror:0.395
[52]	validation_0-merror:0.005	validation_1-merror:0.38
[53]	validation_0-merror:0.005	validation_1-merror:0.39
[54]	validation_0-merror:0.005	validation_1-merror:0.39
[55]	validation_0-merror:0.005	validation_1-merror:0.375
[56]	validation_0-merror:0.005	validation_1-merror:0.39
[57]	validation_0-merror:0.005	validation_1-merror:0.38
[58]	validation_0-merror:0.005	validation_1-merror:0.39
[59]	validation_0-merror:0.00375	validation_1-merror:0.38
[60]	validation_0-merror:0.00375	validation_1-merror:0.38
[61]	validation_0-merror:0.00375	validation_1-merror:0.37
[62]	validation_0-merror:0.00375	validation_1-merror:0.375
[63]	validation_0-merror:0.00375	validation_1-merror:0.37
[64]	validation_0-merror:0.00375	validation_1-merror:0.38

```
[65] validation_0-merror:0.00375 validation_1-merror:0.37
[66] validation_0-merror:0.00375 validation_1-merror:0.375
[67] validation_0-merror:0.0025 validation_1-merror:0.365
[68] validation_0-merror:0.0025 validation_1-merror:0.36
[69] validation_0-merror:0.0025 validation_1-merror:0.365
[70] validation_0-merror:0.0025 validation_1-merror:0.37
[71] validation_0-merror:0.0025 validation_1-merror:0.37
[72] validation_0-merror:0.0025 validation_1-merror:0.375
[73] validation_0-merror:0.0025 validation_1-merror:0.375
[74] validation_0-merror:0.00125 validation_1-merror:0.37
[75] validation_0-merror:0.00125 validation_1-merror:0.365
[76] validation_0-merror:0.00125 validation_1-merror:0.37
[77] validation_0-merror:0.00125 validation_1-merror:0.365
[78] validation_0-merror:0.00125 validation_1-merror:0.365
[79] validation_0-merror:0.00125 validation_1-merror:0.365
[80] validation_0-merror:0.00125 validation_1-merror:0.365
[81] validation_0-merror:0.00125 validation_1-merror:0.37
[82] validation_0-merror:0.00125 validation_1-merror:0.37
[83] validation_0-merror:0.00125 validation_1-merror:0.36
[84] validation_0-merror:0.00125 validation_1-merror:0.36
[85] validation_0-merror:0.00125 validation_1-merror:0.375
[86] validation_0-merror:0.00125 validation_1-merror:0.37
[87] validation_0-merror:0.00125 validation_1-merror:0.37
[88] validation_0-merror:0.00125 validation_1-merror:0.375
[89] validation_0-merror:0.00125 validation_1-merror:0.38
[90] validation_0-merror:0.00125 validation_1-merror:0.38
[91] validation_0-merror:0.00125 validation_1-merror:0.375
[92] validation_0-merror:0.00125 validation_1-merror:0.38
[93] validation_0-merror:0.00125 validation_1-merror:0.38
[94] validation_0-merror:0.00125 validation_1-merror:0.38
[95] validation_0-merror:0.00125 validation_1-merror:0.375
[96] validation_0-merror:0.00125 validation_1-merror:0.375
[97] validation_0-merror:0.00125 validation_1-merror:0.37
[98] validation_0-merror:0.00125 validation_1-merror:0.37
[99] validation_0-merror:0.00125 validation_1-merror:0.37
```

```
Out[175]: XGBClassifier(eval_metric='merror', max_bin=128, max_depth=4, n_jobs=6,
               objective='multi:softprob', reg_lambda=0.1)
```

```
In [176]: print(accuracy_score(y_train, XGBC_model.predict(X_train_scaled)))
          print(accuracy_score(y_valid, XGBC_model.predict(X_valid_scaled)))
```

```
0.99875
0.63
```

Deep neural network

```
In [11]: # tensorflow.keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import tensorflow_addons as tfa # import tfa which contains metrics for regression
from tensorflow.keras import regularizers

from tensorflow.random import set_seed
set_seed(0)
```

```
In [12]: # one-hot encode target for neural network
OH_encoder = OneHotEncoder(handle_unknown='ignore', sparse=False)
```

```
In [13]: y_train_OH = OH_encoder.fit_transform(y_train)
          y_valid_OH = OH_encoder.transform(y_valid)
```

```
In [14]: X_train_scaled.shape[1]
```

```
Out[14]: 37
```

```
In [62]: nn_model = keras.Sequential([
            layers.Dense(units=256, input_shape=[X_train_scaled.shape[1]], activation='relu'), # use relu
            layers.Dense(units=128, activation='relu'),
            layers.Dense(units=64, activation='relu'),
```

```
layers.Dense(units=num_class, activation='softmax')
])
```

```
In [63]: optimizer = tf.keras.optimizers.Adam(learning_rate=0.0005)
nn_model.compile(
    optimizer=optimizer,
    loss="categorical_crossentropy", # categorical cross entropy
    metrics='categorical_accuracy' # categorical accuracy as metric
)
```

```
In [51]: nn_model.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param #
=====		
dense_15 (Dense)	(None, 256)	9728
dense_16 (Dense)	(None, 128)	32896
dense_17 (Dense)	(None, 64)	8256
dense_18 (Dense)	(None, 10)	650
=====		
Total params: 51,530		
Trainable params: 51,530		
Non-trainable params: 0		

```
In [64]: history = nn_model.fit(X_train_scaled, y_train_OH,
                                validation_data=(X_valid_scaled, y_valid_OH),
                                epochs = 70)
```

```
Epoch 1/70
25/25 [=====] - 1s 12ms/step - loss: 2.2637 - categorical_accuracy: 0.2387 - val_loss: 2.1965 - val_categorical_accuracy: 0.3600
Epoch 2/70
25/25 [=====] - 0s 7ms/step - loss: 2.1297 - categorical_accuracy: 0.3512 - val_loss: 2.0416 - val_categorical_accuracy: 0.3550
Epoch 3/70
25/25 [=====] - 0s 6ms/step - loss: 1.9506 - categorical_accuracy: 0.3600 - val_loss: 1.8452 - val_categorical_accuracy: 0.3700
Epoch 4/70
25/25 [=====] - 0s 6ms/step - loss: 1.7876 - categorical_accuracy: 0.3675 - val_loss: 1.7310 - val_categorical_accuracy: 0.3650
Epoch 5/70
25/25 [=====] - 0s 6ms/step - loss: 1.6741 - categorical_accuracy: 0.3950 - val_loss: 1.6219 - val_categorical_accuracy: 0.4300
Epoch 6/70
25/25 [=====] - 0s 6ms/step - loss: 1.6110 - categorical_accuracy: 0.4087 - val_loss: 1.5658 - val_categorical_accuracy: 0.4600
Epoch 7/70
25/25 [=====] - 0s 7ms/step - loss: 1.5566 - categorical_accuracy: 0.4387 - val_loss: 1.5389 - val_categorical_accuracy: 0.4500
Epoch 8/70
25/25 [=====] - 0s 6ms/step - loss: 1.5163 - categorical_accuracy: 0.4412 - val_loss: 1.5083 - val_categorical_accuracy: 0.4600
Epoch 9/70
25/25 [=====] - 0s 6ms/step - loss: 1.4762 - categorical_accuracy: 0.4512 - val_loss: 1.4493 - val_categorical_accuracy: 0.4650
Epoch 10/70
25/25 [=====] - 0s 7ms/step - loss: 1.4345 - categorical_accuracy: 0.5025 - val_loss: 1.4548 - val_categorical_accuracy: 0.4950
Epoch 11/70
25/25 [=====] - 0s 6ms/step - loss: 1.4054 - categorical_accuracy: 0.4850 - val_loss: 1.3971 - val_categorical_accuracy: 0.5050
Epoch 12/70
25/25 [=====] - 0s 6ms/step - loss: 1.4033 - categorical_accuracy: 0.4950 - val_loss: 1.4194 - val_categorical_accuracy: 0.4650
Epoch 13/70
25/25 [=====] - 0s 6ms/step - loss: 1.4030 - categorical_accuracy: 0.4863 - val_loss: 1.3643 - val_categorical_accuracy: 0.5250
Epoch 14/70
25/25 [=====] - 0s 7ms/step - loss: 1.3466 - categorical_accuracy: 0.5200 - val_loss: 1.3636 - val_categorical_accuracy: 0.5100
```

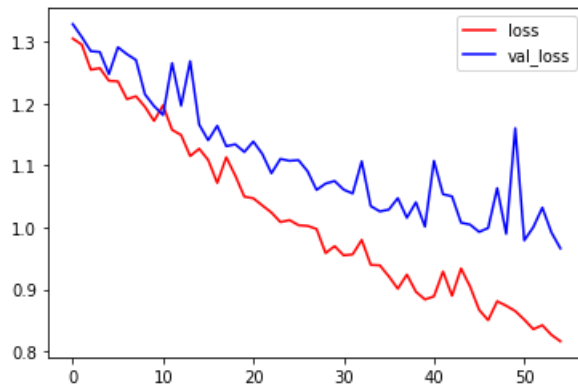
Epoch 15/70
25/25 [=====] - 0s 7ms/step - loss: 1.3219 - categorical_accuracy: 0.5275 - val_loss: 1.3485 - val_categorical_accuracy: 0.4550
Epoch 16/70
25/25 [=====] - 0s 7ms/step - loss: 1.3046 - categorical_accuracy: 0.5238 - val_loss: 1.3279 - val_categorical_accuracy: 0.5050
Epoch 17/70
25/25 [=====] - 0s 8ms/step - loss: 1.2948 - categorical_accuracy: 0.5275 - val_loss: 1.3071 - val_categorical_accuracy: 0.5100
Epoch 18/70
25/25 [=====] - 0s 8ms/step - loss: 1.2544 - categorical_accuracy: 0.5650 - val_loss: 1.2842 - val_categorical_accuracy: 0.5500
Epoch 19/70
25/25 [=====] - 0s 7ms/step - loss: 1.2569 - categorical_accuracy: 0.5387 - val_loss: 1.2829 - val_categorical_accuracy: 0.5650
Epoch 20/70
25/25 [=====] - 0s 6ms/step - loss: 1.2365 - categorical_accuracy: 0.5688 - val_loss: 1.2471 - val_categorical_accuracy: 0.5900
Epoch 21/70
25/25 [=====] - 0s 7ms/step - loss: 1.2356 - categorical_accuracy: 0.5550 - val_loss: 1.2906 - val_categorical_accuracy: 0.5300
Epoch 22/70
25/25 [=====] - 0s 6ms/step - loss: 1.2069 - categorical_accuracy: 0.5813 - val_loss: 1.2794 - val_categorical_accuracy: 0.5400
Epoch 23/70
25/25 [=====] - 0s 7ms/step - loss: 1.2115 - categorical_accuracy: 0.5650 - val_loss: 1.2700 - val_categorical_accuracy: 0.5450
Epoch 24/70
25/25 [=====] - 0s 7ms/step - loss: 1.1946 - categorical_accuracy: 0.5700 - val_loss: 1.2145 - val_categorical_accuracy: 0.5600
Epoch 25/70
25/25 [=====] - 0s 6ms/step - loss: 1.1717 - categorical_accuracy: 0.5763 - val_loss: 1.1957 - val_categorical_accuracy: 0.5750
Epoch 26/70
25/25 [=====] - 0s 6ms/step - loss: 1.1969 - categorical_accuracy: 0.5675 - val_loss: 1.1813 - val_categorical_accuracy: 0.5950
Epoch 27/70
25/25 [=====] - 0s 7ms/step - loss: 1.1574 - categorical_accuracy: 0.5888 - val_loss: 1.2650 - val_categorical_accuracy: 0.5850
Epoch 28/70
25/25 [=====] - 0s 8ms/step - loss: 1.1491 - categorical_accuracy: 0.5987 - val_loss: 1.1963 - val_categorical_accuracy: 0.5850
Epoch 29/70
25/25 [=====] - 0s 6ms/step - loss: 1.1151 - categorical_accuracy: 0.6100 - val_loss: 1.2681 - val_categorical_accuracy: 0.5150
Epoch 30/70
25/25 [=====] - 0s 6ms/step - loss: 1.1269 - categorical_accuracy: 0.6025 - val_loss: 1.1659 - val_categorical_accuracy: 0.5900
Epoch 31/70
25/25 [=====] - 0s 6ms/step - loss: 1.1088 - categorical_accuracy: 0.6275 - val_loss: 1.1405 - val_categorical_accuracy: 0.6250
Epoch 32/70
25/25 [=====] - 0s 6ms/step - loss: 1.0717 - categorical_accuracy: 0.6275 - val_loss: 1.1639 - val_categorical_accuracy: 0.6150
Epoch 33/70
25/25 [=====] - 0s 7ms/step - loss: 1.1131 - categorical_accuracy: 0.6137 - val_loss: 1.1310 - val_categorical_accuracy: 0.6250
Epoch 34/70
25/25 [=====] - 0s 6ms/step - loss: 1.0836 - categorical_accuracy: 0.6325 - val_loss: 1.1341 - val_categorical_accuracy: 0.6550
Epoch 35/70
25/25 [=====] - 0s 6ms/step - loss: 1.0497 - categorical_accuracy: 0.6388 - val_loss: 1.1217 - val_categorical_accuracy: 0.6250
Epoch 36/70
25/25 [=====] - 0s 6ms/step - loss: 1.0470 - categorical_accuracy: 0.6425 - val_loss: 1.1386 - val_categorical_accuracy: 0.6350
Epoch 37/70
25/25 [=====] - 0s 6ms/step - loss: 1.0354 - categorical_accuracy: 0.6463 - val_loss: 1.1184 - val_categorical_accuracy: 0.6350
Epoch 38/70
25/25 [=====] - 0s 6ms/step - loss: 1.0241 - categorical_accuracy: 0.6463 - val_loss: 1.0873 - val_categorical_accuracy: 0.6400
Epoch 39/70
25/25 [=====] - 0s 6ms/step - loss: 1.0086 - categorical_accuracy: 0.6513 - val_loss: 1.1106 - val_categorical_accuracy: 0.6400
Epoch 40/70
25/25 [=====] - 0s 6ms/step - loss: 1.0119 - categorical_accuracy: 0.6600 - val_loss: 1.1074 - val_categorical_accuracy: 0.6200
Epoch 41/70
25/25 [=====] - 0s 8ms/step - loss: 1.0036 - categorical_accuracy: 0.6525 - val_loss: 1.1085 - val_categorical_accuracy: 0.6350

Epoch 42/70
25/25 [=====] - 0s 6ms/step - loss: 1.0025 - categorical_accuracy: 0.6538 - val_loss: 1.0907 - val_categorical_accuracy: 0.6550
Epoch 43/70
25/25 [=====] - 0s 6ms/step - loss: 0.9975 - categorical_accuracy: 0.6575 - val_loss: 1.0605 - val_categorical_accuracy: 0.6350
Epoch 44/70
25/25 [=====] - 0s 8ms/step - loss: 0.9586 - categorical_accuracy: 0.6825 - val_loss: 1.0708 - val_categorical_accuracy: 0.6300
Epoch 45/70
25/25 [=====] - 0s 7ms/step - loss: 0.9699 - categorical_accuracy: 0.6675 - val_loss: 1.0749 - val_categorical_accuracy: 0.6400
Epoch 46/70
25/25 [=====] - 0s 8ms/step - loss: 0.9550 - categorical_accuracy: 0.6775 - val_loss: 1.0610 - val_categorical_accuracy: 0.6550
Epoch 47/70
25/25 [=====] - 0s 8ms/step - loss: 0.9566 - categorical_accuracy: 0.6750 - val_loss: 1.0547 - val_categorical_accuracy: 0.6450
Epoch 48/70
25/25 [=====] - 0s 6ms/step - loss: 0.9799 - categorical_accuracy: 0.6575 - val_loss: 1.1067 - val_categorical_accuracy: 0.6100
Epoch 49/70
25/25 [=====] - 0s 7ms/step - loss: 0.9397 - categorical_accuracy: 0.6787 - val_loss: 1.0345 - val_categorical_accuracy: 0.6800
Epoch 50/70
25/25 [=====] - 0s 7ms/step - loss: 0.9390 - categorical_accuracy: 0.6762 - val_loss: 1.0259 - val_categorical_accuracy: 0.6750
Epoch 51/70
25/25 [=====] - 0s 7ms/step - loss: 0.9212 - categorical_accuracy: 0.6812 - val_loss: 1.0288 - val_categorical_accuracy: 0.6750
Epoch 52/70
25/25 [=====] - 0s 7ms/step - loss: 0.9012 - categorical_accuracy: 0.7000 - val_loss: 1.0473 - val_categorical_accuracy: 0.6550
Epoch 53/70
25/25 [=====] - 0s 7ms/step - loss: 0.9241 - categorical_accuracy: 0.6762 - val_loss: 1.0155 - val_categorical_accuracy: 0.6900
Epoch 54/70
25/25 [=====] - 0s 7ms/step - loss: 0.8964 - categorical_accuracy: 0.7013 - val_loss: 1.0406 - val_categorical_accuracy: 0.6550
Epoch 55/70
25/25 [=====] - 0s 7ms/step - loss: 0.8839 - categorical_accuracy: 0.6938 - val_loss: 1.0014 - val_categorical_accuracy: 0.6750
Epoch 56/70
25/25 [=====] - 0s 6ms/step - loss: 0.8889 - categorical_accuracy: 0.6900 - val_loss: 1.1074 - val_categorical_accuracy: 0.6100
Epoch 57/70
25/25 [=====] - 0s 6ms/step - loss: 0.9289 - categorical_accuracy: 0.6837 - val_loss: 1.0536 - val_categorical_accuracy: 0.6350
Epoch 58/70
25/25 [=====] - 0s 6ms/step - loss: 0.8899 - categorical_accuracy: 0.6988 - val_loss: 1.0501 - val_categorical_accuracy: 0.6200
Epoch 59/70
25/25 [=====] - 0s 7ms/step - loss: 0.9338 - categorical_accuracy: 0.6762 - val_loss: 1.0077 - val_categorical_accuracy: 0.6900
Epoch 60/70
25/25 [=====] - 0s 6ms/step - loss: 0.9047 - categorical_accuracy: 0.6850 - val_loss: 1.0046 - val_categorical_accuracy: 0.6600
Epoch 61/70
25/25 [=====] - 0s 6ms/step - loss: 0.8672 - categorical_accuracy: 0.7025 - val_loss: 0.9927 - val_categorical_accuracy: 0.6800
Epoch 62/70
25/25 [=====] - 0s 7ms/step - loss: 0.8505 - categorical_accuracy: 0.7075 - val_loss: 0.9991 - val_categorical_accuracy: 0.6750
Epoch 63/70
25/25 [=====] - 0s 7ms/step - loss: 0.8810 - categorical_accuracy: 0.7050 - val_loss: 1.0635 - val_categorical_accuracy: 0.6450
Epoch 64/70
25/25 [=====] - 0s 6ms/step - loss: 0.8737 - categorical_accuracy: 0.7038 - val_loss: 0.9897 - val_categorical_accuracy: 0.6750
Epoch 65/70
25/25 [=====] - 0s 6ms/step - loss: 0.8652 - categorical_accuracy: 0.7013 - val_loss: 1.1601 - val_categorical_accuracy: 0.6200
Epoch 66/70
25/25 [=====] - 0s 6ms/step - loss: 0.8514 - categorical_accuracy: 0.6913 - val_loss: 0.9791 - val_categorical_accuracy: 0.6900
Epoch 67/70
25/25 [=====] - 0s 7ms/step - loss: 0.8360 - categorical_accuracy: 0.7312 - val_loss: 1.0004 - val_categorical_accuracy: 0.6750
Epoch 68/70
25/25 [=====] - 0s 7ms/step - loss: 0.8423 - categorical_accuracy: 0.7200 - val_loss: 1.0320 - val_categorical_accuracy: 0.6700

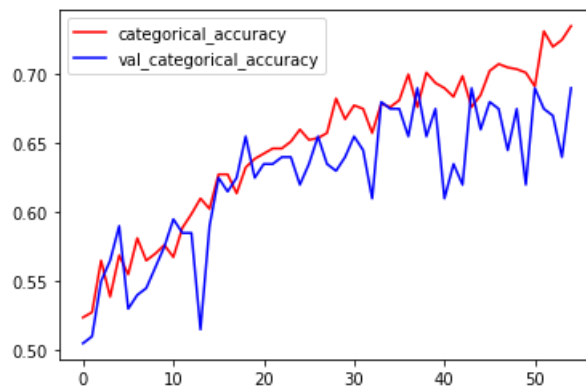
```
Epoch 69/70
25/25 [=====] - 0s 7ms/step - loss: 0.8269 - categorical_accuracy: 0.7250 - val_loss: 0.9922 - val_categorical_accuracy: 0.6400
Epoch 70/70
25/25 [=====] - 0s 7ms/step - loss: 0.8165 - categorical_accuracy: 0.7350 - val_loss: 0.9663 - val_categorical_accuracy: 0.6900
```

```
In [65]: history_df = pd.DataFrame(history.history)
```

```
In [66]: plt.figure()
plt.plot(history.history['loss'][15:], color='red', label='loss')
plt.plot(history.history['val_loss'][15:], color='blue', label='val_loss')
plt.legend();
```



```
In [67]: plt.figure()
plt.plot(history.history['categorical_accuracy'][15:], color='red', label='categorical_accuracy')
plt.plot(history.history['val_categorical_accuracy'][15:], color='blue', label='val_categorical_accuracy')
plt.legend();
```



```
In [68]: nn_model.evaluate(X_train_scaled, y_train_OH)
```

```
25/25 [=====] - 0s 4ms/step - loss: 0.7763 - categorical_accuracy: 0.7475
```

```
Out[68]: [0.7762660384178162, 0.747500023841858]
```

```
In [69]: nn_model.evaluate(X_valid_scaled, y_valid_OH)
```

```
7/7 [=====] - 0s 4ms/step - loss: 0.9663 - categorical_accuracy: 0.6900
```

```
Out[69]: [0.9662549495697021, 0.6899999976158142]
```

Our XGBoost model achieved a validation set accuracy of 0.63. It hugely overfits as the training set accuracy is 0.99875.

Our deep neural network achieved a validation set accuracy of 0.69. It overfits as the training set accuracy is 0.75.

Further hyperparameter tuning could improve the results.

In []:

In []:

In []: